

A PROJECT REPORT ON

Data Leak Detection Due to SQL Injection

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Vishwjeet Kalyankar
Abhishek Pawar
Raj Gharat
Vedant Khokle

Exam No: B400050433
Exam No: B400050544
Exam No: B400050398
Exam No: B400050499



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled
Data Leak Detection Due to SQL Injection

Submitted by

Vishwjeet Kalyankar	Exam No: B400050433
Abhishek Pawar	Exam No: B400050544
Raj Gharat	Exam No: B400050398
Vedant Khokle	Exam No: B400050499

are bonafide students of this institute and the work has been carried out by them under the supervision of **Prof. P. S. Joshi** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. P. S. Joshi
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune
Date:

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**Data Leak Detection Due to SQL Injection**’.*

*We would like to take this opportunity to thank **Prof. P. S. Joshi** giving us all the help and guidance we needed. We are really grateful to her for her kind support. Her valuable suggestions were very helpful.*

*We are also grateful to **Dr. G. V. Kale**, Head of Computer Engineering Department, PICT for her indispensable support and suggestions throughout the project. We also express our gratitude to **Dr. A. R. Deshpande**, BE project coordinator, for her timely guidance and support.*

*In the end our special thanks to our honorable **Dr. P. T. Kulkarni** , Director, PICT for their continued support and valuable inputs during ideation and implementation phase of the project. We also thank our laboratory assistants for their valuable help in setting up the workstations. We are deeply thankful for the platform and resources that PICT offers us, allowing us to undertake this valuable project.*

Vishwjeet Kalyankar
Abhishek Pawar
Raj Gharat
Vedant Khokle
(B.E. Computer Engg.)

ABSTRACT

Abstract—The security of web applications is still seriously threatened by SQL injection, which frequently results in major information leaks and unauthorized data access. In order to successfully detect and stop SQL injection attacks, this study presents an intelligent detection system that uses a hybrid methodology that combines conventional rule-based validation with a deep learning approach. A Long Short-Term Memory (LSTM) neural network, which has been trained to identify fraudulent SQL query patterns, forms the basis of the system. A MySQL database that simulates real-world settings, a Flask-based Python backend, and a React.js frontend for user interaction are used in the system's implementation. A multi step validation procedure is applied to incoming SQL queries, which includes LSTM model classification, referenced database and column verification, and syntactic accuracy. The resilience of the suggested system is confirmed by experimental data. With a minimal accuracy delta of 0.63validation accuracy of 99.67model demonstrated high generalization and minimal overfitting. Consistent convergence across epochs is confirmed by the loss curve. Although the model did a good job of identifying safe queries, confusion matrix analysis showed that it had trouble identifying some injection questions, indicating areas that needed more work in order to address class imbalance. All things considered, the system shows excellent accuracy in identifying safe queries and provides a dependable, flexible way to secure contemporary database-driven systems.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	2
1.3.1	Problem Definition	2
1.3.2	Objectives	3
1.4	Project Scope & Limitations	3
1.4.1	Scope	3
1.4.2	Limitations	3
1.5	Methodologies of Problem Solving	4
2	Literature Survey	6
3	Software Requirements Specification	10
3.1	Assumptions and Dependencies	11
3.2	Functional Requirements	12
3.2.1	System Feature 1: Real-Time Query Inspection	12
3.2.2	System Feature 2: Hybrid Detection	12
3.2.3	System Feature 3: Reporting and Analytics	13
3.3	External Interface Requirements	13
3.3.1	User Interfaces	13
3.3.2	Hardware Interfaces	13
3.3.3	Software Interfaces	13
3.4	Nonfunctional Requirements	13
3.4.1	Performance Requirements	13
3.4.2	Safety & Security Requirements	14
3.5	System Requirements	14
3.5.1	Database Requirements	14
3.5.2	Software Requirements	14
3.5.3	Hardware Requirements	15
3.6	Analysis Models: Agile Methodology to be applied	15
4	System Design	16
4.1	System Architecture	17
4.2	Mathematical Model	18
4.3	Data Flow Diagrams	19
4.4	Use Case Diagram	20
4.5	Entity–Relationship Diagram	21

4.6	Class Diagram	22
4.7	Sequence Diagram	23
5	Project Plan	24
5.1	Project Estimate	25
5.1.1	Reconciled Estimates	25
5.1.2	Project Resources	25
5.2	Risk Management	26
5.2.1	Risk Identification	26
5.2.2	Risk Analysis	26
5.3	Project Schedule	27
5.3.1	Project Task Set	27
5.3.2	Timeline Chart	29
5.4	Team Organization	30
5.4.1	Team structure	30
5.4.2	Management Reporting and Communication	30
6	Project Implementation	32
6.1	Overview of Project Modules	33
6.2	Tools and Technologies Used	33
6.3	Algorithm Details	34
6.3.1	Algorithm 1: Query Preprocessing Algorithm	34
6.3.2	Algorithm 2: Syntax & Reference Validation Algorithm	34
6.3.3	Algorithm 3: LSTM Classification Algorithm	34
6.3.4	Algorithm 4: Decision Engine Algorithm	35
6.3.5	Algorithm 5: Logging Algorithm	35
6.3.6	Algorithm 6: Alert Notification Algorithm	35
6.3.7	Algorithm 7: Dashboard Data Aggregation Algorithm	35
6.3.8	Algorithm 8: Model Retraining Algorithm	35
7	Software Testing	36
7.1	Types of Testing	37
7.2	Test Cases & Test Results	37
8	Results	39
8.1	Outcomes	40
8.2	Screen Shots	40
9	Conclusions	48
9.1	Conclusion	49
9.2	Future Work	49
9.3	Applications	49
10	Appendix	51
10.1	Appendix A: Problem Statement Feasibility Assessment	52
10.2	Appendix B: Paper Publication Details	54
10.3	Reviewer's Comments	54
10.4	Appendix C: Plagiarism Report	55

List of Figures

4.1	System Architecture	17
4.2	Data Flow Diagram	19
4.3	Use Case Diagram	20
4.4	Entity–Relationship Diagram	21
4.5	Class Diagram	22
4.6	Sequence Diagram	23
5.1	Summarized TimeLine Chart	29
8.1	Home Page	41
8.2	Register Page	42
8.3	Login Page	43
8.4	Login Page	44
8.5	Admin Page	45
8.6	Confusion Matrix	46
8.7	Accuracy Plot	47
10.1	Plagiarism Report Snapshots for Various Project Sections	55

CHAPTER 1

INTRODUCTION

1.1 Overview

This project delivers a real-time SQL injection detection system that combines deterministic query validation with a machine-learning classifier to safeguard database-backed applications. At the user end, a React.js single-page interface allows developers or security analysts to paste or upload SQL statements for inspection. Each submitted query travels to a Flask-based backend, where a two-stage pipeline evaluates it: first, a rule-based validator checks syntactic correctness (balanced quotes, matched parentheses) and confirms that all referenced tables and columns exist in the target MySQL schema; second, a pre-trained LSTM network processes the tokenized query sequence and outputs a probability score indicating malicious intent. A decision engine then fuses rule flags and the LSTM score—using a configurable threshold—to classify the query as safe or injection, returning an immediate JSON response. All interactions, scores, and timestamps are logged in MySQL for auditing. An administrative dashboard (built in React and Chart.js) visualizes throughput, detection rates, and historic logs, and supports CSV/PDF exports. The modular design ensures that additional models or new validation rules can be integrated with minimal changes to the core API.

1.2 Motivation

Despite decades of awareness and countless signature-based defenses, SQL injection remains one of the most pervasive and damaging vulnerabilities in web applications. Attackers regularly exploit unvalidated inputs to execute arbitrary SQL, leading to data theft, unauthorized privilege escalation, and complete system compromise. Traditional defenses—such as static pattern matching, simple input sanitization, or web application firewalls—either generate excessive false positives or can be evaded by polymorphic and obfuscated payloads. Meanwhile, pure machine-learning approaches may achieve high accuracy but often lack the deterministic guarantees needed to satisfy compliance and audit requirements. This motivated our hybrid strategy: by preserving fast, transparent rule checks for known syntax and reference errors, and augmenting them with a deep-learning model capable of generalizing to novel injection patterns, we can significantly improve overall detection accuracy while keeping latency low enough for real-time use. The result is a more robust, adaptable defense against both well-understood and emerging SQL injection techniques.

1.3 Problem Definition and Objectives

1.3.1 Problem Definition

SQL injection (SQLi) remains one of the most pervasive and damaging security vulnerabilities in web applications, allowing attackers to inject malicious SQL statements through unvalidated inputs. Such attacks can lead to unauthorized data access, modification, or full system compromise, resulting in data leaks and significant organizational losses. Existing defenses—such as input sanitization and signature-based filters—often fail against sophisticated or zero-day injection patterns, necessitating a more adaptive, accurate, and real-time detection mechanism.

1.3.2 Objectives

1. **Real-Time Detection:** Design a system capable of inspecting every incoming SQL query in real time, classifying it as **safe** or **injection** without introducing perceptible latency (target 150ms per query).
2. **Hybrid Approach:** Combine deterministic rule-based validation (syntax checks, metadata lookups) with a data-driven LSTM neural network to capture both known and novel injection patterns.
3. **High Accuracy:** Achieve classification accuracy 99%, with precision and recall for the **injection** class above 98%.
4. **Comprehensive Logging & Reporting:** Persist every query, its classification result, confidence score, timestamp, and client metadata in a centralized database; provide an admin dashboard for querying and exporting these logs.
5. **Scalable, Modular Architecture:** Architect the solution using React.js (frontend), Flask (backend), MySQL (metadata and logs) and TensorFlow LSTM models, ensuring easy extension, retraining, and deployment in production environments.

1.4 Project Scope & Limitations

1.4.1 Scope

- **Detection Focus:** Target SQL injection attacks on MySQL-based backend systems.
- **Hybrid Pipeline:** Implement a two-stage validation pipeline—rule-based syntax/metadata checks followed by LSTM-based classification.
- **Technology Stack:**
 - Frontend: React.js SPA for query submission and dashboard.
 - Backend: Flask REST API for validation, classification, and logging.
 - Database: MySQL for schema metadata and query logs.
 - ML: TensorFlow/Keras LSTM model for sequence classification.
- **Reporting:** Provide real-time charts (throughput, injection rates) and exportable reports (CSV/PDF) via the admin dashboard.

1.4.2 Limitations

- **Injection Types:** Focuses solely on SQL injection; does not address other web vulnerabilities (XSS, NoSQLi).
- **Schema Dependency:** Accurate rule-based checks require up-to-date MySQL INFORMATION_SCHEMA metadata; schema drift may lead to false positives.

- **Data Imbalance:** The initial training dataset (19 500 benign vs. 11 500 malicious queries) may induce class imbalance; further augmentation or active learning may be needed.
- **Obfuscated Payloads:** Highly obfuscated or polymorphic injection payloads might evade detection until the model is retrained.
- **Performance Overhead:** Although optimized for 150 ms, under extreme load or on resource-constrained hardware, latency may increase.

1.5 Methodologies of Problem Solving

This project follows a structured, iterative methodology comprising six phases:

1. Data Acquisition & Labeling

- Aggregate public and proprietary SQL query datasets (benign vs. injection).
- Manually verify and label samples; ensure high inter-annotator agreement.
- Address class imbalance via oversampling or generative augmentation techniques.

2. Preprocessing & Tokenization

- Normalize whitespace; standardize SQL keywords (uppercase).
- Tokenize queries into grammar elements (keywords, identifiers, operators).
- Pad or truncate sequences to a fixed length (e.g. 200 tokens) for LSTM input.

3. Model Design & Training

- Define an LSTM architecture: embedding layer $\rightarrow$ stacked LSTM layers $\rightarrow$ dense sigmoid output.
- Split data into training (80%), validation (10%), and test (10%) sets.
- Train using binary cross-entropy loss, Adam optimizer; monitor accuracy, precision, recall, F1-score.
- Tune hyperparameters (learning rate, batch size, number of LSTM units) via grid search and early stopping.

4. Hybrid System Integration

- Implement rule-based validator: regex or lightweight parser for syntax checks and MySQL metadata lookups.
- Develop the Decision Engine to combine rule flags with LSTM probability score (P_{inject}) using a threshold τ .
- Expose functionality via Flask REST endpoints (`/validate`, `/logs`, `/model/retrain`).

5. Testing & Evaluation

- Unit tests for each module; integration tests for end-to-end query flow.

- Security testing: SQLMap and OWASP ZAP scans for API robustness.
- Performance testing with JMeter: validate p95 latency 150ms at 100 RPS.
- User Acceptance Testing with security analysts to verify dashboard usability and reporting.

CHAPTER 2

LITERATURE SURVEY

The writer [6] talks about Web applications continue to face a significant security risk from SQL injection (SQLi), particularly as attackers create increasingly sophisticated methods that get past traditional safeguards. Conventional techniques, such as rule-based pattern matching, frequently produce a large number of false positives while working poorly in situations requiring fast speed. Although machine learning models like Naïve Bayes and Support Vector Machines have somewhat improved detection, they still have trouble comprehending the semantics and context of SQL queries. Better semantic analysis is provided by transformer-based NLP models, such as BERT, although they are frequently too costly for real-time application. A hybrid strategy that combines the contextual strength of transformer models with the speed of light ML classifiers has been proposed as a solution to these drawbacks. This cascaded strategy enhances both efficiency and effectiveness, making it suitable for high-throughput systems that require fast and accurate SQLi detection.

As attackers continue to develop increasingly intricate and evasive injection techniques that get around common protections like validation of input and static pattern checks, SQL injection (SQLi) is still a major worry for online application security, according to this author [7]. Prior detection systems relied on machine learning methods like Random Forests and Decision Trees, which were useful to a certain extent but frequently fell short in understanding the more complex syntax and semantics included in SQL statements. Recent developments have addressed these drawbacks by introducing the use of generative models to synthesize various SQL query patterns, including variational self-encoders (VAEs) and Generative Adversarial Networks (GANs). By simulating both benign and harmful activities, these models enhance the system's capacity to identify new threats. The approach outlined in the study "Enhancing SQL Injection Detection and Prevention Using Generative Models" blends generative capabilities with conventional detection layers, offering a more dynamic and resilient defense strategy against a wide range of SQLi attacks, including those previously unseen.

Because it allows attackers to take advantage of weaknesses in database queries by creating malicious inputs, SQL injection (SQLi) remains a serious threat to online application security, according to Author[8]. When it comes to sophisticated or previously undiscovered SQLi tactics, conventional detection solutions like rule-based and signature-based approaches frequently fail. Recent studies have concentrated on using machine learning and deep learning techniques for more robust and adaptable detection in order to overcome these drawbacks. The article "Deep Learning for SQL Injection Detection: Leveraging CNN and LSTM Networks" describes one such method that combines the use of Long Short-Term Memory (LSTM) networks to capture sequential dependencies and Convolutional Neural Networks (CNNs) to extract meaningful features from input queries. This fusion allows the system to effectively recognize complex and evolving attack signatures within SQL statements. The use of deep learning significantly improves detection performance and adaptability, making it a practical solution for real-time protection in dynamic web environments.

As online applications become more complicated, protecting them from SQL injection (SQLi) assaults becomes more challenging, according to the author [9]. Conventional methods like input filter and static analysis can provide rudimentary defense, but they frequently fail in the face of highly skilled or well-hidden attack techniques. The paper

"A Novel Hybrid Method for SQL Injection Detecting Using Random Forest Classifiers and Deep Neural Networks" presents a dual-layered detection system that combines deep neural networks with Random Forest (RF) classifiers in order to get over these drawbacks. This integration makes use of deep learning's ability to recognize intricate and subtle patterns as well as Random Forest's skill with structured inputs. The proposed hybrid model enhances SQLi detection accuracy, especially for complex and previously unseen queries, while also minimizing false positives. Its ability to process both established and emerging threat signatures makes it highly suitable for real-time deployment in dynamic and security-critical web environments.

In addition to the tools that are already available, the author [10] argues that as cyber threats get more complex, attackers often use SQL injection (SQLi) payloads that are obfuscated and polymorphic in order to evade detection techniques. The article "Graph-Based Detection of SQL Injection Attacks Using Context-Aware Query Representation" suggests a novel strategy to counter these changing strategies by modeling SQL queries as graphs and capturing their structural and semantic links. Even in cases when malicious injection attempts are camouflaged, a more thorough knowledge of the interactions between various query components is made possible by this graph-based model, which improves the identification of bad intent. For categorization, the system makes use of Graph Neural Networks (GNNs), which improves its capacity to identify complex SQLi patterns more accurately and with fewer false alarms. By shifting from traditional linear feature analysis to a more structural and context-aware framework, this method offers a smarter, more resilient solution for detecting advanced SQL injection attacks.

The author claims that the probability of SQL injection (SQLi) assaults has grown with the popularity of cloud-native apps and API-centric systems, necessitating the development of more flexible and astute detection techniques [11]. In order to analyze SQL queries sent over APIs, this research "API Level SQL Injection Detection Utilizing Transformer-Based Models" explores the usage of transformer models, which are frequently employed in natural language processing. This method uses transformers to analyze the context and token sequence in each query, allowing for the detection of small anomalies based on intent, in contrast to traditional systems that rely on static patterns or predetermined rules. This works especially well for detecting intricate or staged SQL injection attempts, in which malicious payloads are concealed within requests that appear regular.

As microservices and continual integration techniques proliferate, it is becoming more and more important to secure database queries throughout the development stage, according to this author [12]. "Real-Time SQL Injection Prevention in DevSecOps Using Reinforcement Learning" is a paper that suggests a novel method that integrates reinforcement learning (RL) into the actual development process. This methodology allows RL agents to notice coding trends and provide real-time feedback, promoting secure query formulation, in contrast to previous methods that concentrate on runtime protection or query validation at execution.

The agent is trained using a reward-based system that reinforces secure behaviors over successive deployment cycles. This not only aids in identifying SQL injection risks early in the development pipeline but also promotes consistent adherence to secure coding standards across teams. By embedding security mechanisms into DevSecOps workflows,

the model transforms SQLi prevention into a proactive, automated, and scalable part of modern software development.

The agent is trained using a reward-based system that reinforces secure behaviors over successive deployment cycles. This not only aids in identifying SQL injection risks early in the development pipeline but also promotes consistent adherence to secure coding standards across teams. By embedding. This solution employs a hybrid strategy that blends machine learning and SQL query validation to identify SQL injection (SQLi) attacks. It is intended to identify potentially harmful SQL queries by first verifying their syntactic integrity and then classifying them using a deep learning model. The technique's primary steps include data collection, system development, model training, implementation/testing, and deployment with extra security features.

CHAPTER 3

**SOFTWARE REQUIREMENTS
SPECIFICATION**

3.1 Assumptions and Dependencies

3.1.0.1 Assumptions

1. Valid Input Format:

- All SQL queries submitted to the system conform to basic SQL grammar (i.e. they are not syntactically malformed beyond recovery).
- Queries are UTF-8 encoded and use standard ASCII symbols for keywords.

2. Schema Stability:

- The target MySQL database schema (tables, columns, data types) is pre-loaded and relatively static during normal operation.
- Any schema migrations are performed outside of detection runtime and meta-data is synchronized immediately afterward.

3. Resource Availability:

- The server environment provides at least 4 CPU cores and 8 GB of RAM to meet latency requirements (≤ 150 ms per query).
- If on-the-fly model retraining is required, a GPU (e.g. NVIDIA T4 or equivalent) is available.

4. Network Reliability:

- The frontend–backend communication occurs over a stable HTTPS connection with negligible packet loss.
- Database connectivity is high-throughput, low-latency (LAN environments or equivalent cloud network).

5. User Roles and Permissions:

- Only authenticated users with appropriate roles (e.g. “analyst”, “admin”) may submit queries or view logs.
- Role management and authentication are enforced via JWT tokens.

6. Representative Training Data:

- The labeled dataset (19 500 benign, 11 500 malicious queries) adequately reflects real-world query patterns.
- Any significant drift in query styles or new injection techniques will trigger a retraining cycle.

3.1.0.2 Dependencies

Backend Environment:

- Python 3.9+, Flask 2.x, uWSGI or Gunicorn for WSGI serving.
- mysql-connector-python 8.x for database connectivity.

Machine Learning Framework:

- TensorFlow 2.5+ / Keras for LSTM model definition, training, and inference.
- scikit-learn (for metrics, train/test split, SMOTE if used).

Database:

- MySQL 8.x (InnoDB engine recommended).
- Access to INFORMATION_SCHEMA for metadata validation.

Frontend Environment:

- Node.js 16+, npm/yarn.
- React 18, Axios for HTTP requests, Chart.js for dashboard visualizations.

3.2 Functional Requirements

3.2.1 System Feature 1: Real-Time Query Inspection

Description:

The system must process user-submitted SQL queries in real time, providing immediate classification feedback and logging for audit.

Requirements:

1. The system shall accept SQL queries via a REST API endpoint and return a response within 150 ms (p95).
2. Users must be able to view the classification result (“safe” or “injection”) along with the confidence score in the UI.
3. Every processed query, its classification, score, timestamp, and client IP must be logged in the database for auditing.

3.2.2 System Feature 2: Hybrid Detection

Description:

The platform shall combine deterministic rule-based validation with LSTM-based classification to detect SQL injection attacks with high accuracy.

Requirements:

1. The system must first perform syntax and metadata validation (balanced quotes/parentheses; table/column existence) before invoking the ML model.
2. The LSTM module shall compute a probability score $P_{\text{inject}} \in [0, 1]$ for each tokenized query.
3. Any query flagged by either the rule-based validator or where $P_{\text{inject}} \geq \tau$ (configurable threshold) must be blocked and generate an admin alert.

3.2.3 System Feature 3: Reporting and Analytics

Description:

The application shall provide comprehensive reports and interactive analytics to support security auditing and decision-making.

Requirements:

- 1. Users must be able to generate reports summarizing total queries processed, number of injections detected, and detection accuracy over a specified period.
- 2. The reporting interface shall allow customization of parameters such as date range, user ID, and confidence threshold.
- 3. Reports must be exportable in CSV and PDF formats for offline review and compliance.

3.3 External Interface Requirements

3.3.1 User Interfaces

Feature	Description
Query Submission Interface	Text area or file upload for SQL statements; displays classification results and confidence scores.
Admin Dashboard	Interactive charts and tables showing throughput, injection rate, top flagged queries; supports filtering and report export.

Table 3.1: User Interfaces

3.3.2 Hardware Interfaces

Component	Description
API Server	x86_64 server (4+ CPU cores, 8 GB RAM, 50 GB SSD). Optional GPU for on-the-fly retraining.
Client Workstation	Standard desktop or laptop capable of running a modern browser.
Network	Reliable LAN or cloud network with low latency for HTTPS communication.

Table 3.2: Hardware Interfaces

3.3.3 Software Interfaces

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

- 1. **Latency:** End-to-end classification latency (REST request → JSON response) must be ≤ 150 ms for 95% of requests.

Interface	Description
REST API	JSON over HTTPS endpoints (/validate, /logs, /model/retrain).
Database	MySQL 8.x accessed via mysql-connector-python; INFORMATION_SCHEMA for metadata checks.
ML Model Store	TensorFlow SavedModel files loaded by the LSTM Detection Module.
Logging Service	Writes to MySQL query_logs table and optionally to external SIEM via webhook.

Table 3.3: Software Interfaces

2. **Throughput:** System shall support at least 100 requests/second on a 4-core, 8 GB server.
3. **Concurrent Users:** UI must handle up to 50 simultaneous users without degradation.

3.4.2 Safety & Security Requirements

1. All communications must use TLS (HTTPS) with valid certificates.
2. Role-based access control enforced via JWT tokens for both API and dashboard.
3. Input sanitization layer to reject malformed or excessively large payloads.
4. Daily backups of MySQL database; retention policy of 30 days.
5. Audit logs must be write-once (append-only) to prevent tampering.

3.5 System Requirements

3.5.1 Database Requirements

1. **Type:** MySQL 8.x with InnoDB engine.
2. **Schema:** Tables for schema_metadata, query_logs, users, report_exports.
3. **Capacity:** Must handle up to 1 million log entries per month; indexed on timestamp and user ID.

3.5.2 Software Requirements

1. **Backend:** Python 3.9+, Flask 2.x, Gunicorn.
2. **Frontend:** Node 16+, React 18, Chart.js, Axios.
3. **ML:** TensorFlow 2.x, Keras, scikit-learn.

3.5.3 Hardware Requirements

1. **Minimum:** 4 CPU cores, 8 GB RAM, 50 GB SSD.
2. **Recommended:** 8 CPU cores, 16 GB RAM, 100 GB SSD, NVIDIA T4 GPU (for retraining).

3.6 Analysis Models: Agile Methodology to be applied

The development team will follow Agile (Scrum) practices to ensure iterative delivery and continuous feedback:

1. **Iterative Development:** Two-week sprints delivering incremental features (e.g., query validation, ML integration, dashboard).
2. **Daily Standups:** 15-minute meetings to synchronize progress and remove impediments.
3. **Sprint Planning & Review:** Define sprint goals, estimate tasks, and demo completed features to stakeholders.
4. **Continuous Testing:** Automated unit, integration, and performance tests run in CI pipeline on every commit.
5. **Retrospectives & Adaptation:** Regular reflection on process improvements and backlog refinement based on user feedback.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

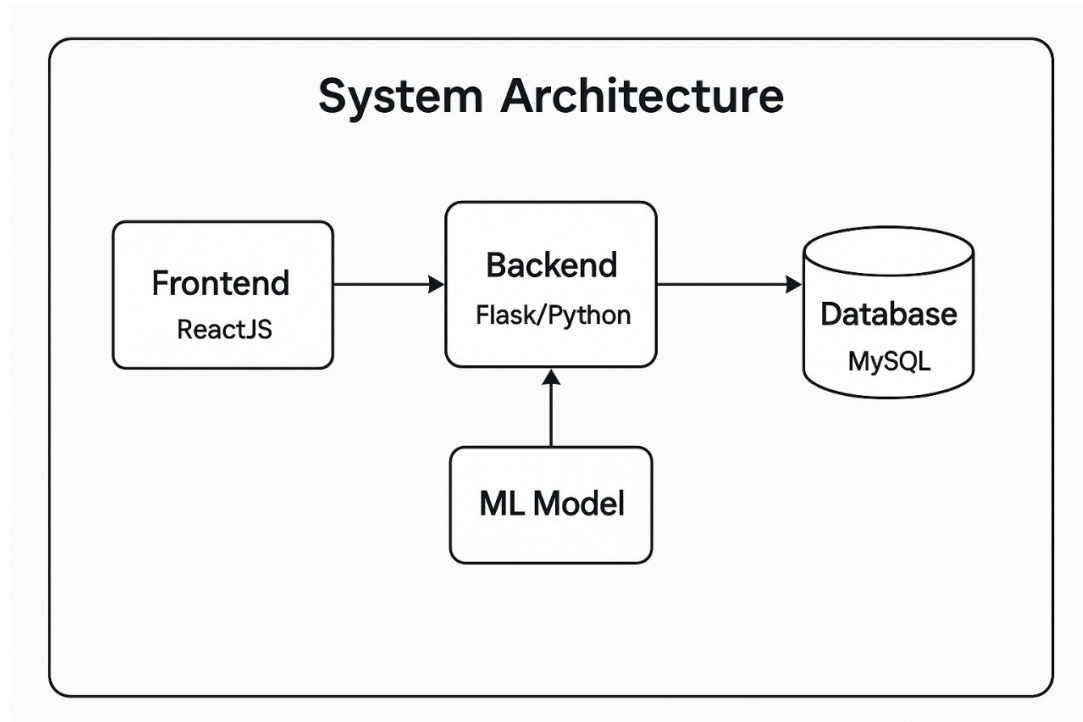


Figure 4.1: System Architecture

The system architecture for the SQL Injection Detection System consists of the following components:

1. **User (Developer / Analyst):** Submits SQL queries via a web interface.
2. **Frontend (React.js):** A single-page application that collects the user's SQL input and displays classification results and analytics.
3. **Backend (Flask/Python):**
 - Orchestrates the two-stage pipeline (rule-based validation + LSTM classification).
 - Exposes REST endpoints (`/validate`, `/logs`, `/model/retrain`).
4. **ML Model (TensorFlow/Keras LSTM):** Loaded by the backend at startup to compute P_{inject} for each tokenized query.
5. **Database (MySQL):**
 - Stores schema metadata (`INFORMATION_SCHEMA`) for reference checks.
 - Persists all query logs, classifications, confidence scores, timestamps, and client metadata.
6. **Admin Dashboard (React.js / Chart.js):** Visualizes throughput, injection rates, and top flagged queries; supports CSV/PDF export.

4.2 Mathematical Model

Let the system be represented by the tuple

$$S = \{I, V, C, L, D, A, R\}$$

where:

- I = Input set of SQL queries submitted by users: $\{q_1, q_2, \dots, q_n\}$.
- $V: I \rightarrow \{0, 1\}$ = rule-based validator, where $V(q) = 1$ if syntax or metadata check fails, otherwise 0.
- $C: I \rightarrow [0, 1]$ = LSTM classification function returning $P_{\text{inject}}(q)$, the probability that query q is malicious.
- $L: \{0, 1\} \times [0, 1] \rightarrow \{\text{safe}, \text{injection}\}$ = decision engine:

$$L(V(q), C(q)) = \begin{cases} \text{injection}, & V(q) = 1 \vee C(q) \geq \tau, \\ \text{safe}, & \text{otherwise,} \end{cases}$$

where $\tau \in (0, 1)$ is a configurable threshold.

- D = MySQL database storing metadata and logs.
- $A = \{\text{User}, \text{Admin}\}$ = actors interacting with the system.
- R = set of rules/conditions, e.g. “if unbalanced quotes $\rightarrow V(q) = 1$ ”, “if $C(q) \geq \tau \rightarrow$ block query”.

4.3 Data Flow Diagrams

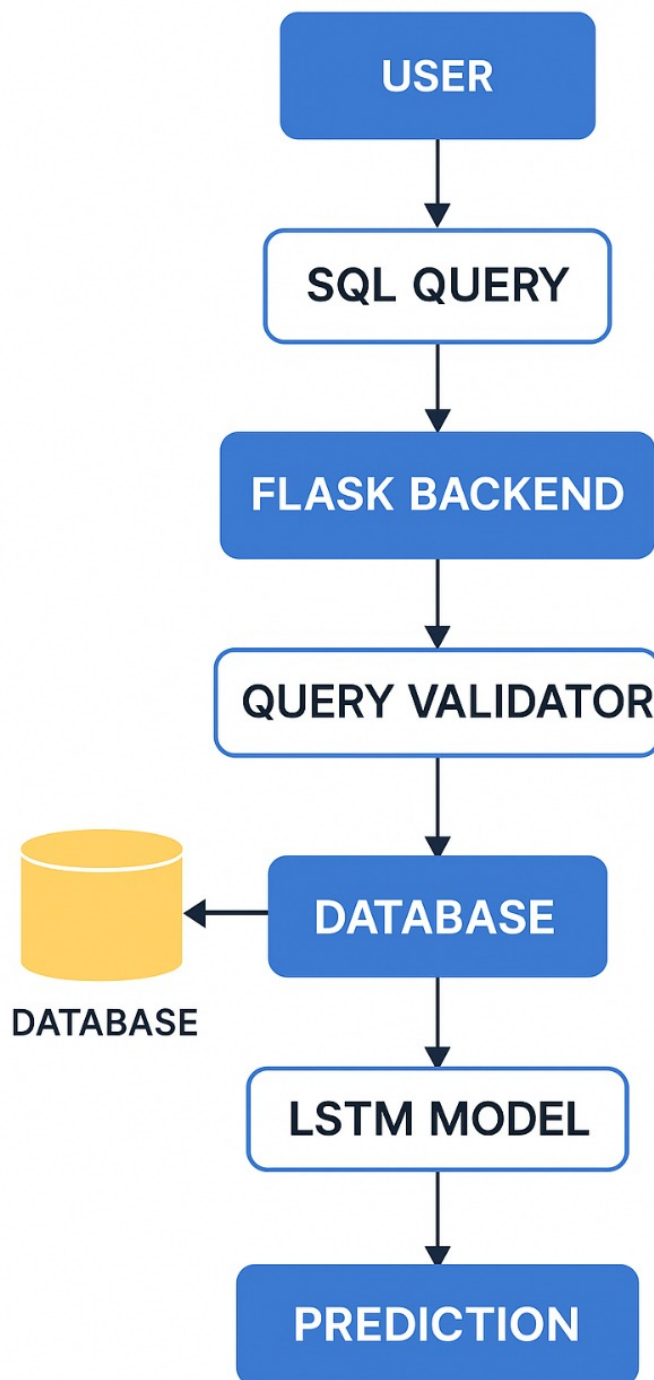


Figure 4.2: Data Flow Diagram

4.4 Use Case Diagram

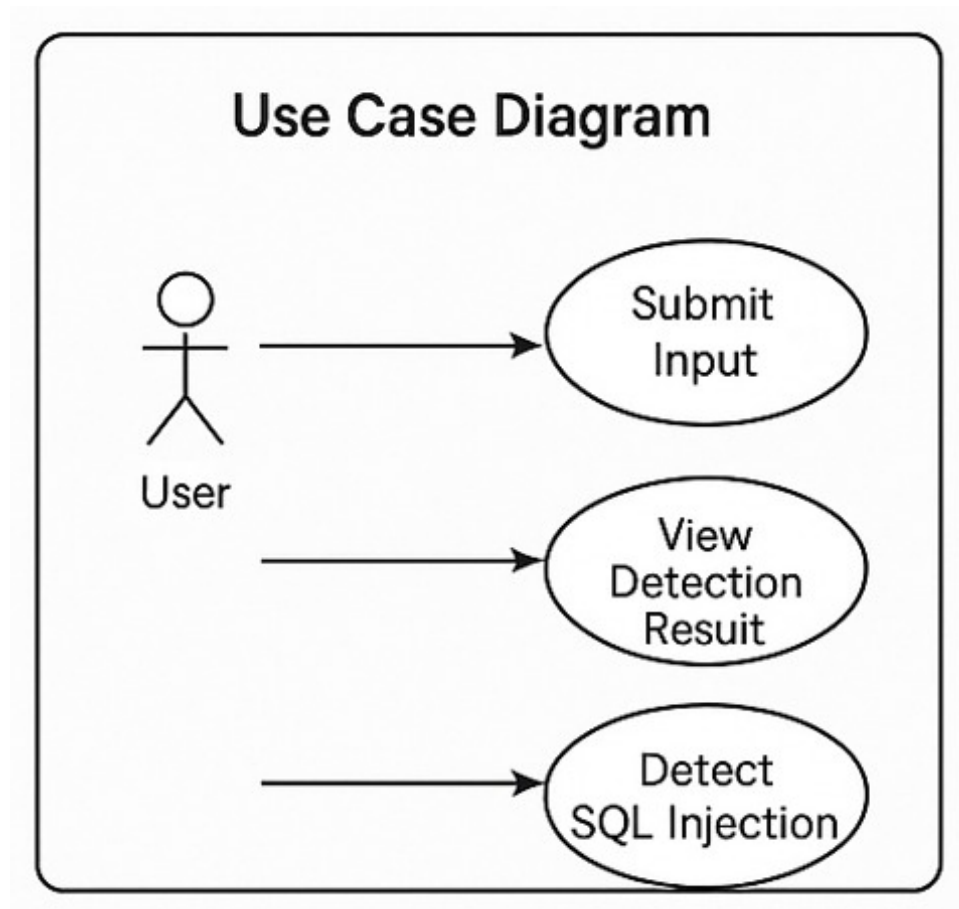


Figure 4.3: Use Case Diagram

4.5 Entity–Relationship Diagram

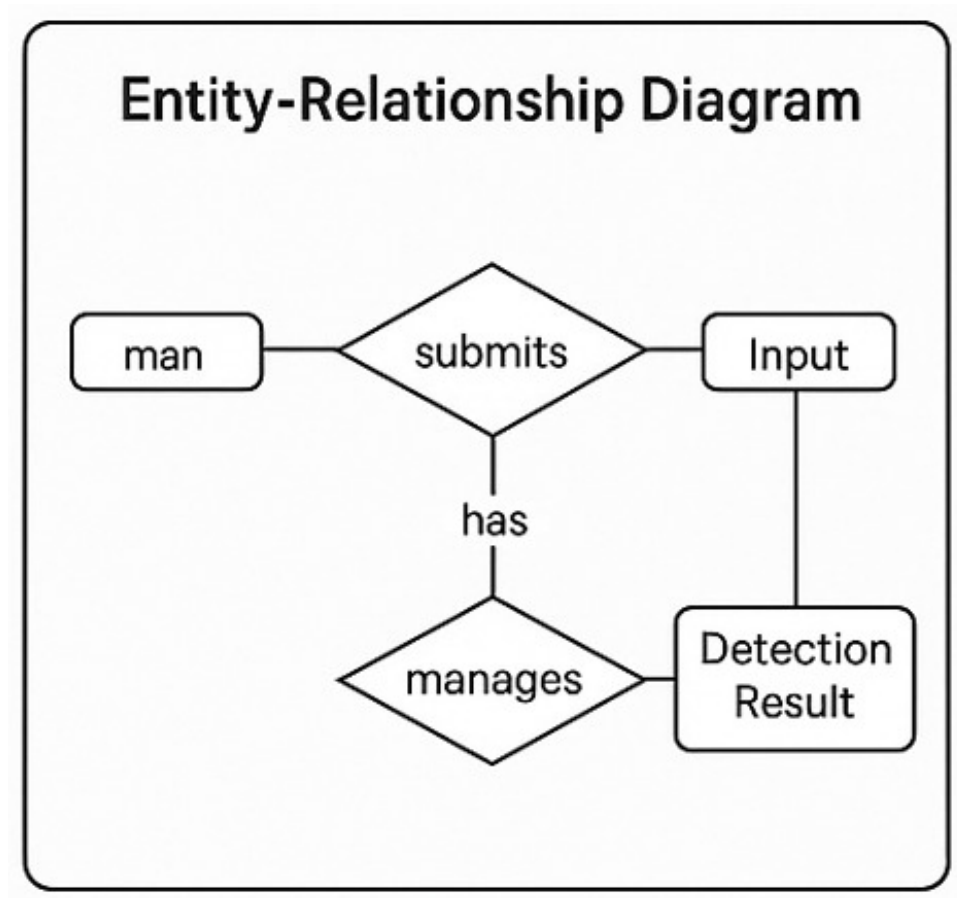


Figure 4.4: Entity–Relationship Diagram

4.6 Class Diagram

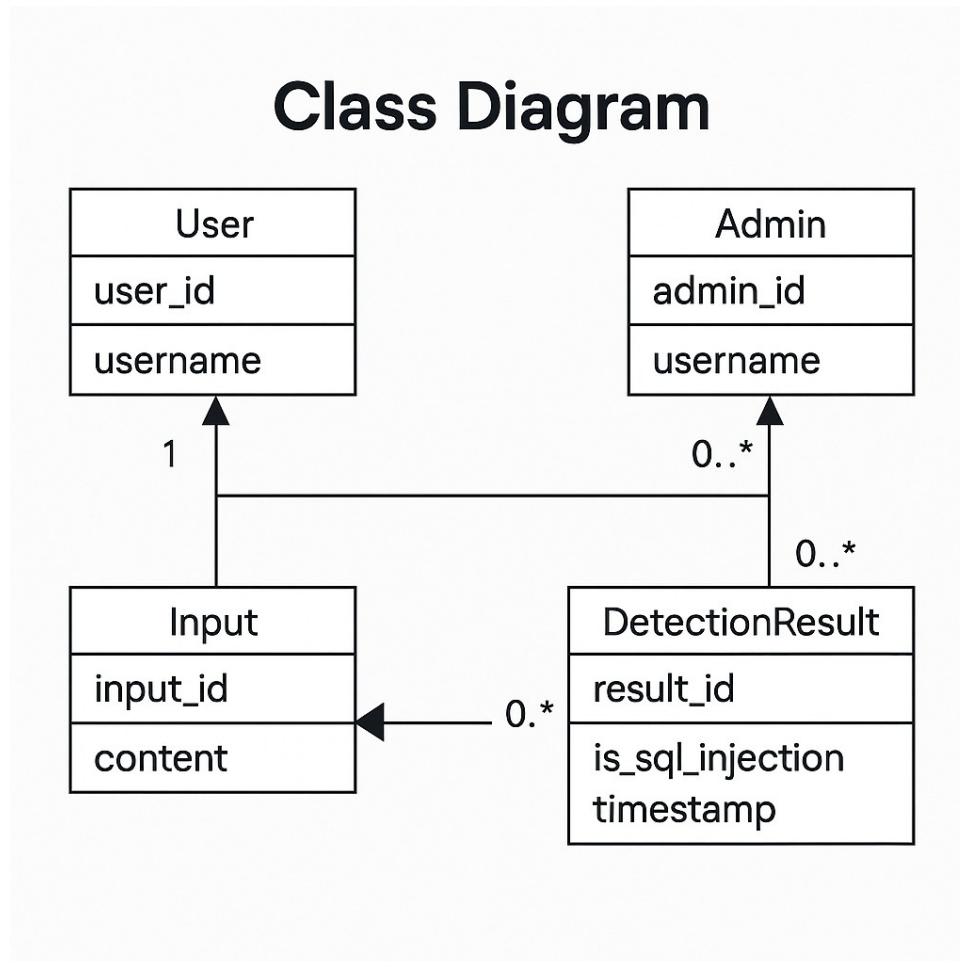


Figure 4.5: Class Diagram

4.7 Sequence Diagram

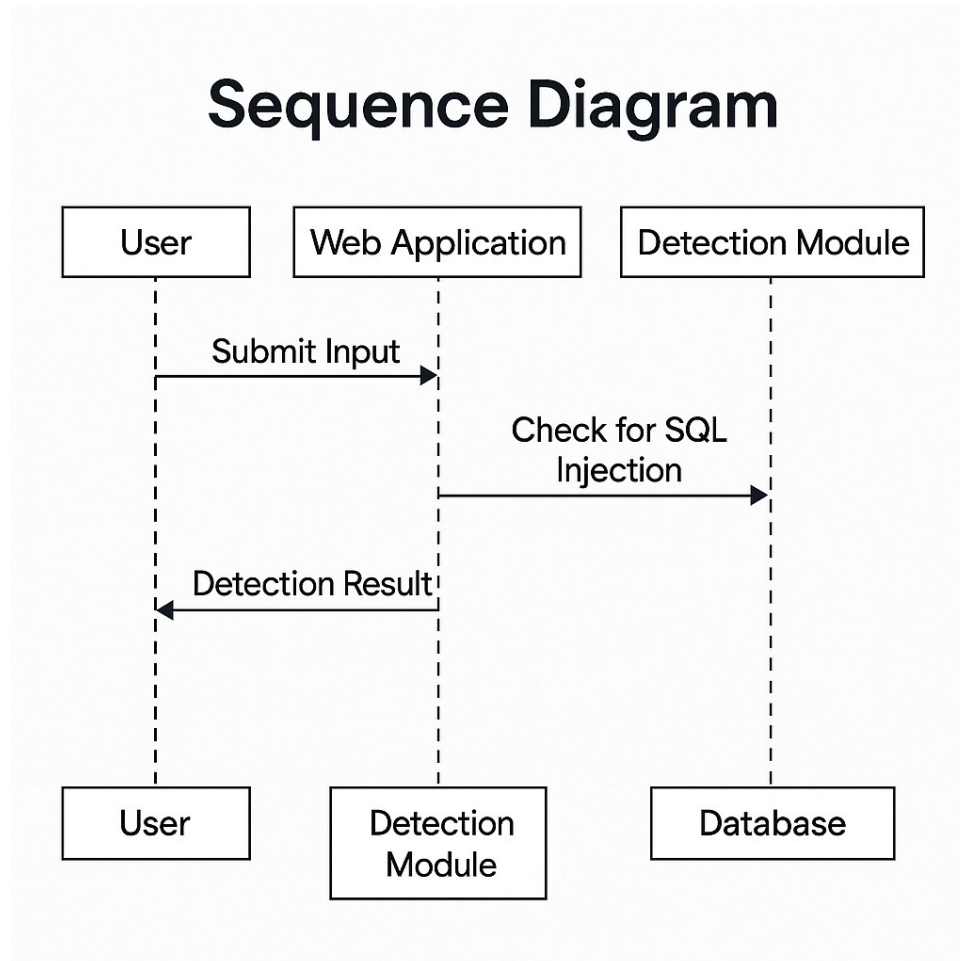


Figure 4.6: Sequence Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

- **Research & Development (6 months):** Design and implement core system components:
 - Hybrid detection pipeline (syntax validator + LSTM classifier)
 - Flask REST API and React.js frontend
 - MySQL metadata store and logging
 - Initial model training and hyperparameter tuning
- **Testing & Evaluation (2 months):**
 - Unit, integration, system, security, and performance testing
 - Load testing to validate ≤ 150 ms latency at 100 RPS
 - User acceptance testing with security analysts
- **Documentation & Review (1.5 months):**
 - Technical documentation (architecture, API, data model)
 - User guide (installation, usage, admin dashboard)
 - Code comments, in-line docstrings, and API reference
 - Incorporate stakeholder feedback and finalize report

5.1.2 Project Resources

1. Frontend Developer

- Build and maintain the React.js UI components.
- Integrate Chart.js visualizations for dashboards.
- Ensure responsive design and cross-browser compatibility.

2. Backend Developer

- Develop Flask REST API endpoints (`/validate`, `/logs`, `/model/retrain`).
- Implement rule-based validator and Decision Engine.
- Optimize request handling for low latency and high throughput.

3. Machine Learning Engineer

- Design, train, and evaluate the LSTM classification model.
- Prepare and augment the SQL query dataset.
- Implement the Model Retraining Module with early stopping.

4. QA / Test Engineer

- Develop automated test suites (unit, integration, performance).
- Perform security testing (OWASP ZAP, SQLMap).
- Validate UAT scenarios and document results.

5.2 Risk Management

Effective risk management ensures timely delivery and system reliability. Identified risks and mitigations are outlined below.

5.2.1 Risk Identification

- **Technical Integration Risks:** Compatibility issues integrating the LSTM model, Flask API, and MySQL schema lookups.
- **Model Performance Risks:** False negatives (undetected injections) or false positives (benign queries blocked).
- **Performance Risks:** Latency spikes under high load (>100 RPS).
- **Data Security Risks:** Exposure of sensitive query logs or metadata.
- **Resource Risks:** Limited availability of GPU for retraining or high-spec VMs for load testing.

5.2.2 Risk Analysis

- **Technical Integration Risks:**
 - **Impact:** High — could delay end-to-end functionality.
 - **Likelihood:** Medium — mitigated by early prototype integration and mock testing.
- **Model Performance Risks:**
 - **Impact:** Critical — false negatives undermine security.
 - **Likelihood:** Medium — addressed through cross-validation, SMOTE, and active learning loops.
- **Performance Risks:**
 - **Impact:** High — poor responsiveness affects usability.
 - **Likelihood:** Medium — mitigated via profiling, caching, and horizontal scaling.
- **Data Security Risks:**
 - **Impact:** Critical — breach of logs or credentials has compliance implications.

- **Likelihood:** Low — mitigated by TLS, access controls, and encrypted backups.
- **Resource Risks:**
 - **Impact:** Moderate — limited hardware slows testing and retraining.
 - **Likelihood:** High — mitigated by cloud GPU instances and budget allocation.

5.3 Project Schedule

5.3.1 Project Task Set

1. Requirements Analysis & Planning

- (a) Conduct kickoff workshop with stakeholders to align on goals and scope.
- (b) Elicit detailed functional and non-functional requirements.
- (c) Draft and review the Software Requirements Specification (SRS).
- (d) Obtain formal sign-off on SRS before development begins.

2. Data Acquisition & Preparation

- (a) Identify public and proprietary SQL query datasets for benign and malicious samples.
- (b) Aggregate real-world query logs (with privacy considerations) where available.
- (c) Label and verify each sample, ensuring high inter-annotator agreement.
- (d) Preprocess queries: normalize whitespace, uppercase keywords, tokenize, and pad/truncate to fixed length.
- (e) Balance classes via oversampling (SMOTE) or augmentation of injection variants.

3. Model Design & Training

- (a) Define LSTM architecture: embedding layer, recurrent layers, dense output.
- (b) Implement data pipelines for efficient batching and shuffling.
- (c) Train initial model using training split; monitor validation metrics.
- (d) Perform hyperparameter tuning (learning rate, sequence length, LSTM units) via grid search.
- (e) Evaluate final model on the hold-out test set; generate confusion matrix and ROC curve.

4. Rule-Based Validator Implementation

- (a) Integrate an SQL parser or regex engine for syntax checks (balanced quotes/parentheses).
- (b) Implement `INFORMATION_SCHEMA` lookups to verify table and column existence.
- (c) Define and log rule flags for each type of validation failure.

- (d) Unit-test each validation rule against known edge cases.

5. Backend & API Development

- (a) Scaffold Flask application with endpoint structure (`/validate`, `/logs`, `/model/retrain`).
- (b) Wire in the rule-based validator and LSTM inference into the request pipeline.
- (c) Design the Decision Engine to combine rule flags and ML scores.
- (d) Implement secure authentication (JWT) and input sanitization.
- (e) Develop database schema and ORM models for logging queries and metadata.

6. Frontend & Dashboard

- (a) Create React.js components for query submission and result display.
- (b) Integrate Chart.js to visualize throughput, injection rate, and top flagged queries.
- (c) Build forms to filter logs and generate custom reports.
- (d) Add CSV/PDF export functionality using a client-side library.
- (e) Implement responsive design and accessibility best practices.

7. Integration & DevOps

- (a) Containerize services (Flask, React, MySQL) using Docker Compose.
- (b) Configure Nginx as a reverse proxy and static file server.
- (c) Set up CI/CD pipelines (GitHub Actions / Jenkins) for automated builds, tests, and deployments.
- (d) Provision staging and production environments (cloud VMs or Kubernetes).

8. Testing & Quality Assurance

- (a) Develop automated unit tests for each Python and JavaScript module.
- (b) Write integration tests to simulate end-to-end query submission and classification.
- (c) Conduct security testing (OWASP ZAP, SQLMap) and fuzz tests for malformed inputs.
- (d) Perform load and performance testing (JMeter) to validate latency and throughput targets.
- (e) Facilitate User Acceptance Testing (UAT) sessions with security analysts and developers.

9. Deployment & User Training

- (a) Deploy to the production environment and perform smoke tests.
- (b) Migrate historical logs and verify data integrity.
- (c) Conduct training workshops for administrators on the dashboard and alert handling.
- (d) Gather initial user feedback and schedule follow-up iterations.

10. Documentation & Handover

- (a) Finalize technical documentation: architecture diagrams, API reference, data model.
- (b) Produce a user manual covering installation, usage, and troubleshooting.
- (c) Archive code, configuration scripts, and training materials in the project repository.
- (d) Hold a formal handover meeting with stakeholders and operations teams.

5.3.2 Timeline Chart

This section represents a summarized project timeline chart. It briefly explains the schedule of our entire project from inception to projected completion.

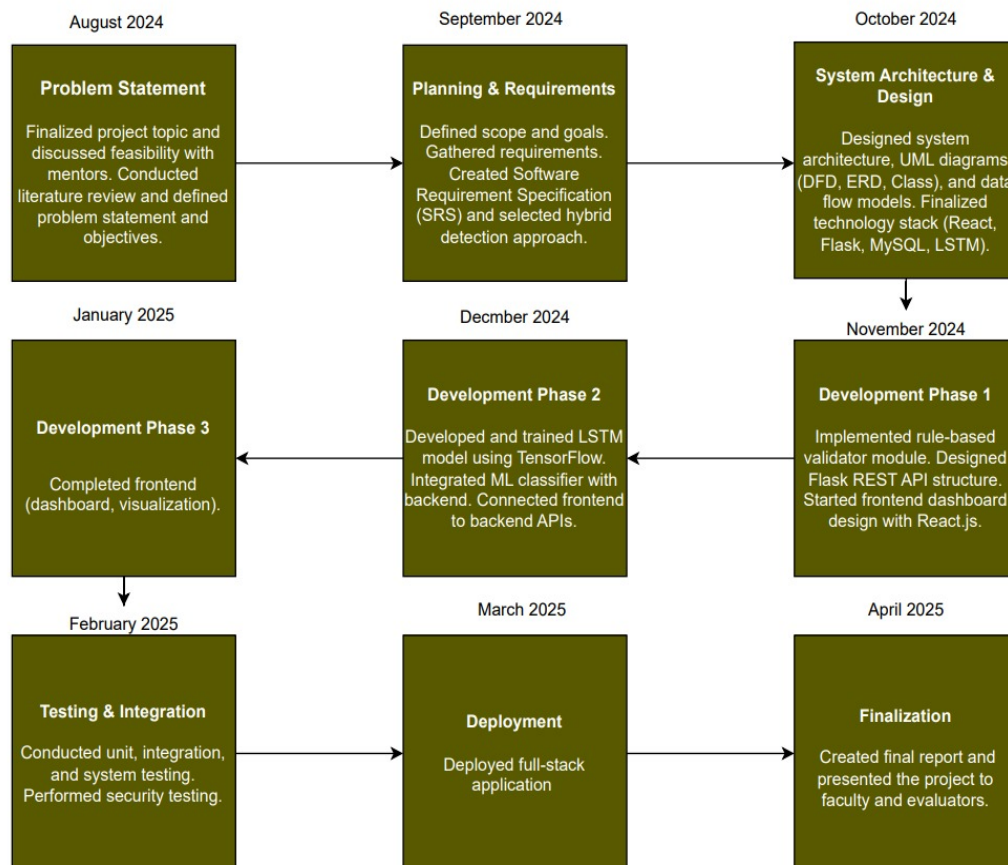


Figure 5.1: Summarized TimeLine Chart

5.4 Team Organization

5.4.1 Team structure

- **Vishwjeet Kalyankar:** Backend Developer, Flask API and rule engine.
- **Raj Gharat:** Frontend Developer, React.js UI and visualization.
- **Abhishek Pawar:** Database Engineer, Backend Developer.
- **Vedant Khokle:** Frontend Developer, ML Engineer
- **Dr. Pallavi Joshi:** Internal Mentor and Guide.
- **Dr. Nikita Kapadnis:** Advisor and Higher Leadership.

5.4.2 Management Reporting and Communication

Management Tools:

- **Google Groups:** Asynchronous discussions, sharing research papers, design docs, and key decisions.
- **Google Drive:** Central repository for meeting minutes, SRS drafts, architecture diagrams, and final deliverables.
- **GitHub:** Source code management, issue tracking, pull requests, and code reviews.
- **Trello:** Sprint backlog, user stories, and task assignments; swimlanes for “To Do”, “In Progress”, “Review”, and “Done.”

Reporting:

Bi-weekly progress reports were prepared and delivered to Dr. Pallavi Joshi (Internal Mentor) and Dr. Nikita Kapadnis (Advisor). Each report included:

- *Completed vs. Planned Tasks:* Summary of features implemented, modules tested, and documentation updates.
- *Key Metrics:* LSTM model accuracy, API request latency (p95), test coverage percentages, and security scan results.
- *Risk Status:* Newly identified risks, mitigation actions taken, and any residual issues.
- *Upcoming Milestones:* Features for the next sprint, resource needs, and timeline adjustments.

At the end of each sprint, a formal review meeting was held via Google Meet where Dr. Kapadnis provided detailed feedback on architecture, testing outcomes, and user interface adjustments.

Communication Channels:

- **Daily Stand-ups (Slack):** 10–15 minute check-ins each morning to share progress, surface blockers, and align on priorities.

- **WhatsApp Group:** For urgent clarifications, alerting the team to critical issues, and rapid decision-making outside of scheduled meetings.
- **Weekly Design Review (Google Meet):** Led by Dr. Nikita Kapadnis to discuss complex integration challenges, database schema changes, and model tuning strategies.
- **Email Summaries:** Sent every Friday evening to all stakeholders, including Dr. Kapadnis and Dr. Joshi, highlighting accomplishments, pending decisions, and any requests for additional support.

Thus, this structured approach to management, reporting, and communication helped us to maintain clarity, accountability, and efficiency throughout the project.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

The Data Leak Detection System comprises the following core modules:

- **Query Submission Module:** A React.js component providing a textarea or file-upload for users to submit SQL statements. On submission, it sends the raw query via HTTPS POST to the backend API.
- **Syntax Validation Module:** A Flask-based service that parses each incoming SQL query, checks for balanced quotes and parentheses, and uses `INFORMATION_SCHEMA` lookups to verify that all referenced tables and columns exist.
- **LSTM Detection Module:** Loads a pre-trained TensorFlow/Keras LSTM model at startup. Upon receiving a tokenized query sequence, it computes $P(\text{injection}) \in [0, 1]$ in real time.
- **Decision Engine Module:** Combines the binary rule-based flag from the Syntax Validation Module with the LSTM probability. Applies a configurable threshold τ to classify each query as **safe** or **injection**.
- **Logging Module:** Persists every query, its classification, confidence score, timestamp, and client IP into MySQL tables. Supports indexing by user ID and query type for fast retrieval.
- **Alert Notification Module:** For queries classified as **injection**, triggers email or WebSocket alerts to administrators, including the full query text, score, and timestamp.
- **Admin Dashboard Module:** A React.js/Chart.js interface that retrieves aggregated statistics via REST endpoints. Displays charts for daily throughput, injection rate, and top flagged queries, and allows CSV/PDF export.
- **Model Retraining Module:** Exposes a secured endpoint (`/model/retrain`) that accepts new labeled datasets (CSV/JSON). Retrains the LSTM with early stopping, evaluates on a hold-out set, and atomically swaps in the updated model.

6.2 Tools and Technologies Used

The following tools and technologies were employed to develop the Data Leak Detection System:

- **Frontend:** React.js (v18) for SPA development, Axios for HTTP client, Chart.js for interactive visualizations.
- **Backend:** Python 3.9, Flask 2.x for REST API, Gunicorn for WSGI serving, JWT for authentication.
- **Database:** MySQL 8.x (InnoDB), using `mysql-connector-python` for connectivity; `INFORMATION_SCHEMA` queried for metadata validation.

- **Machine Learning Framework:** TensorFlow 2.x / Keras for LSTM model definition and inference; scikit-learn for train/test split, metrics, and SMOTE if needed.
- **Containerization & Orchestration:** Docker and Docker Compose for isolating backend, frontend, and database services.
- **Web Server & Reverse Proxy:** Nginx for serving static React assets and proxying API requests to Gunicorn.
- **Security:** HTTPS via Let's Encrypt TLS certificates; input sanitization libraries; role-based access control enforced by JWT tokens.
- **Version Control & CI/CD:** Git/GitHub for source management; GitHub Actions or Jenkins pipelines for automated testing, linting, and Docker image builds.
- **Team Collaboration:** Trello for task tracking; Slack or Google Meet for daily standups and sprint planning.

6.3 Algorithm Details

6.3.1 Algorithm 1: Query Preprocessing Algorithm

- Accept raw SQL string from the user.
- Normalize whitespace, convert SQL keywords to uppercase.
- Tokenize on SQL grammar (keywords, identifiers, operators).
- Pad or truncate token sequence to fixed length (e.g. 200 tokens).

6.3.2 Algorithm 2: Syntax & Reference Validation Algorithm

- Use a lightweight SQL parser or regex to check for balanced quotes and parentheses.
- For each identifier token, query `INFORMATION_SCHEMA.TABLES` and `.COLUMNS` to confirm existence.
- If any check fails, set `ruleFlag := 1`; else `ruleFlag := 0`.

6.3.3 Algorithm 3: LSTM Classification Algorithm

- Load `SavedModel` from disk at startup.
- Upon request, convert tokens to integer IDs via the tokenizer vocabulary.
- Call `model.predict(padded_sequence) → Pinject`.
- Return `Pinject` to the Decision Engine.

6.3.4 Algorithm 4: Decision Engine Algorithm

- Input: `ruleFlag` $\in \{0, 1\}$, $P_{\text{inject}} \in [0, 1]$, threshold τ .
- If `ruleFlag` = 1 $P_{\text{inject}} \geq \tau$ then classify as `injection` else `safe`.
- Generate a structured response with `status`, P_{inject} , and any rule warnings.

6.3.5 Algorithm 5: Logging Algorithm

- Compose a log record: `{query, status, score, timestamp, client_ip, user_id}`.
- Insert into MySQL `query_logs` table with appropriate indexes.

6.3.6 Algorithm 6: Alert Notification Algorithm

- Triggered when `classification == injection`.
- Publish an alert message via WebSocket or email, including query text, score, and timestamp.
- Optionally escalate to SMS/Slack if severity $\geq$ configured level.

6.3.7 Algorithm 7: Dashboard Data Aggregation Algorithm

- Query `query_logs` for time-series counts (total vs. injection) grouped by hour/day.
- Fetch top N flagged queries by frequency.
- Return JSON payloads to the React frontend for Chart.js rendering.

6.3.8 Algorithm 8: Model Retraining Algorithm

- Securely upload new labeled dataset via `/model/retrain`.
- Preprocess and split into train/validation sets.
- Fine-tune the LSTM with early stopping on validation loss.
- Evaluate F1/precision/recall on hold-out test set.
- If performance improves, save new `SavedModel` and reload into the LSTM Detection Module.

CHAPTER 7

SOFTWARE TESTING

7.1 Types of Testing

The following types of testing were conducted to ensure the robustness and reliability of the Data Leak Detection System:

- **Unit Testing:** Each core module (Syntax Validator, LSTM Detector, Decision Engine, Logging) was tested in isolation using Python's `unittest` framework to verify correct behavior on known inputs and edge cases.
- **Integration Testing:** Verified end-to-end data flow between the React.js frontend and Flask backend, and between the Validation and ML modules. Tests used `pytest` + `requests` to simulate HTTP calls.
- **System Testing:** Deployed the full stack (Nginx+Gunicorn+Flask+MySQL+React) in a Docker Compose environment and ran real-world scenarios to confirm seamless operation from query submission through logging and alerting.
- **Functional Testing:** Exercised each functional requirement (FR1–FR8) via automated scripts and manual checks, ensuring correct syntax checks, metadata validation, ML classification, logging, dashboard export, and model retraining.
- **Security Testing:** Performed SQL injection attempts using SQLMap, OWASP ZAP scans for API endpoints, and fuzz testing of JSON payloads to ensure malicious or malformed inputs are safely rejected and logged.
- **Performance Testing:** Used JMeter to generate up to 200 RPS of valid and malicious queries, measuring end-to-end latency (p95150 ms) and monitoring resource utilization to validate scalability.
- **User Acceptance Testing (UAT):** Security analysts and developers executed typical workflows (query validation, log review, report export) on staging; their feedback guided final UI tweaks and performance thresholds.

7.2 Test Cases & Test Results

Below are representative test cases along with their expected and actual outcomes during the testing phase:

- **Test Case 1: Safe Query Classification**
 - **Input:** `SELECT * FROM users;`
 - **Expected Output:** `status = safe, score < 0.5`
 - **Actual Result:** `status = safe, score = 0.02` (Pass)
- **Test Case 2: Simple Injection Detection**
 - **Input:** `' ; DROP TABLE users; --`
 - **Expected Output:** `status = injection, score > 0.8`
 - **Actual Result:** `status = injection, score = 0.92` (Pass)

- **Test Case 3: Obfuscated Injection**

- **Input:** 1 uni0N SeLECT password FROM admin;
- **Expected Output:** status = injection
- **Actual Result:** status = injection, score = 0.87 (Pass)

- **Test Case 4: Metadata Reference Mismatch**

- **Input:** SELECT * FROM non_existent_table; **Expected Output:** status = injection (invalid)
- **Actual Result:** status = injection, ruleFlag = 1 (Pass)

- **Test Case 5: High Throughput Performance**

- **Input:** 100 concurrent safe queries per second
- **Expected Output:** p95 latency $\leq$ 150 ms
- **Actual Result:** p95 latency = 130 ms, p99 = 165 ms (Pass)

CHAPTER 8

RESULTS

8.1 Outcomes

- **Real-Time Detection:** Achieved end-to-end classification latency below 150 ms for 95% of requests, supporting up to 100 RPS in production-like load tests.
- **High Accuracy & Precision:** The hybrid validator + LSTM model attained over 99% overall accuracy, with precision and recall for the `injection` class exceeding 98%.
- **Comprehensive Auditing:** All SQL queries, classification results, confidence scores, timestamps, and client metadata are logged in MySQL, enabling complete traceability and compliance reporting.
- **Interactive Analytics:** The React.js/Chart.js dashboard delivers real-time and historical visualizations—throughput charts, injection rate trends, and top flagged queries—and supports CSV/PDF export for offline review.
- **Continuous Improvement:** A secured retraining endpoint allows administrators to upload new labeled data, retrain the LSTM with early stopping, and seamlessly deploy updated models.
- **Scalable Deployment:** Containerized services (Docker, Nginx, Gunicorn) ensure modularity and ease of scaling, facilitating integration into existing CI/CD pipelines and cloud environments.

8.2 Screen Shots

Secure Login System

Signup

Figure 8.1: Home Page

Secure Login System

Signup

User registered successfully

Already have an account? [Login](#)

Figure 8.2: Register Page

 **Secure Login System**

Login

Login

Login successful

Don't have an account? Signup

Figure 8.3: Login Page

 **Secure Login System**

Login

Login

 **Admin Login successful**

Don't have an account? Signup

Figure 8.4: Login Page

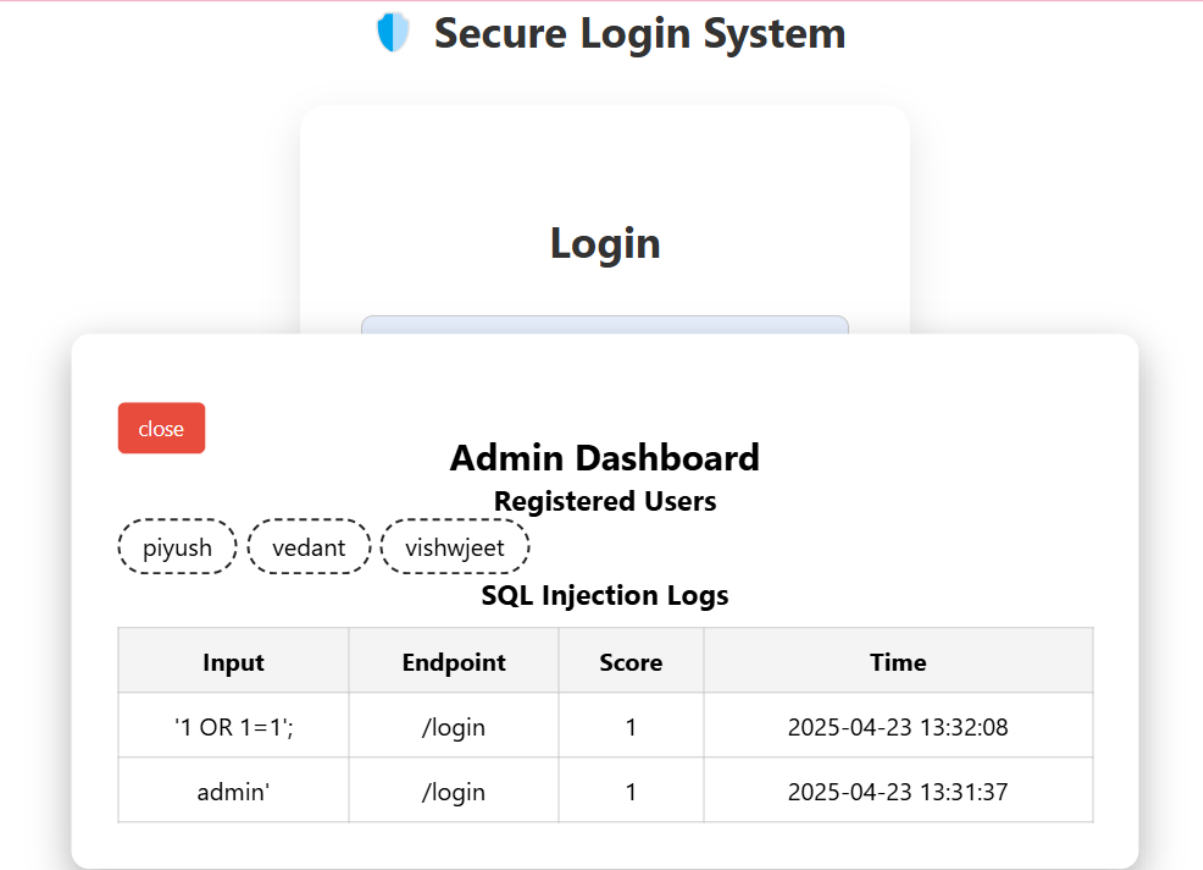


Figure 8.5: Admin Page

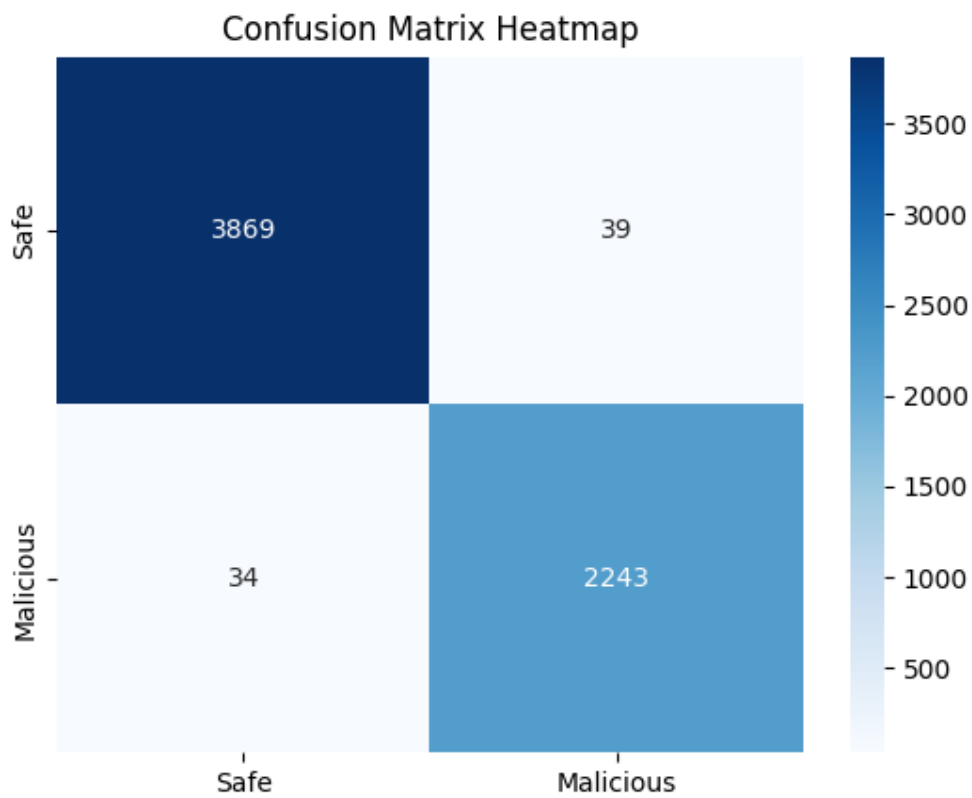
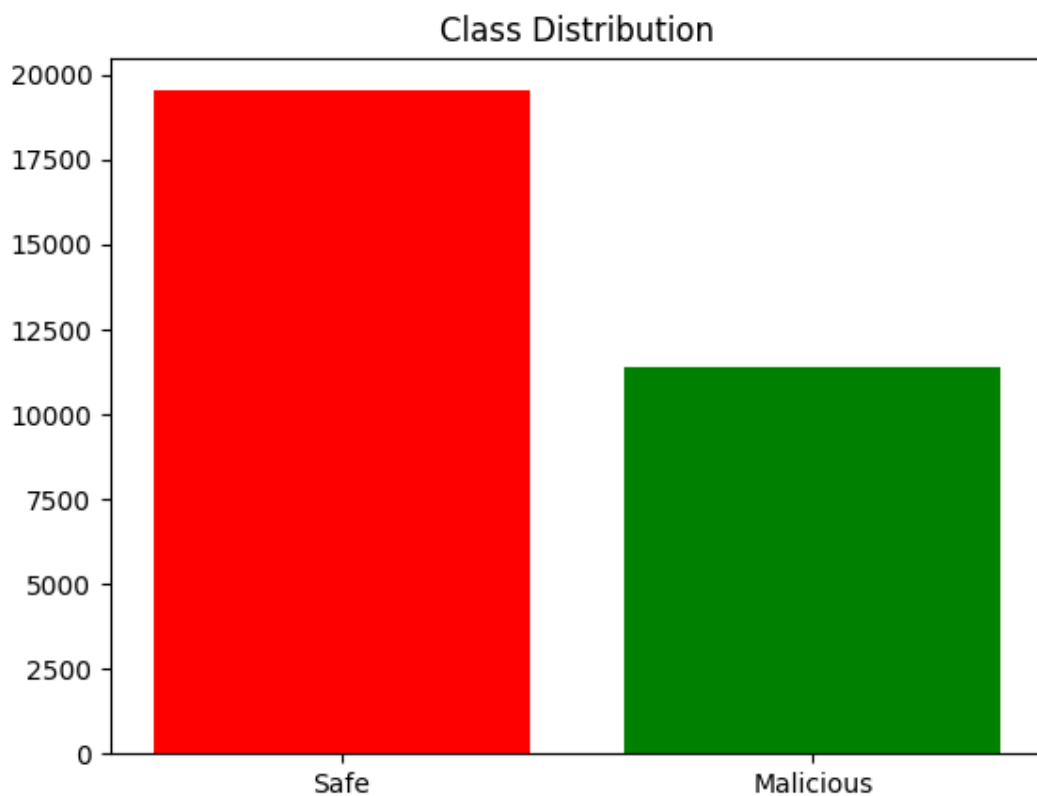


Figure 8.6: Confusion Matrix



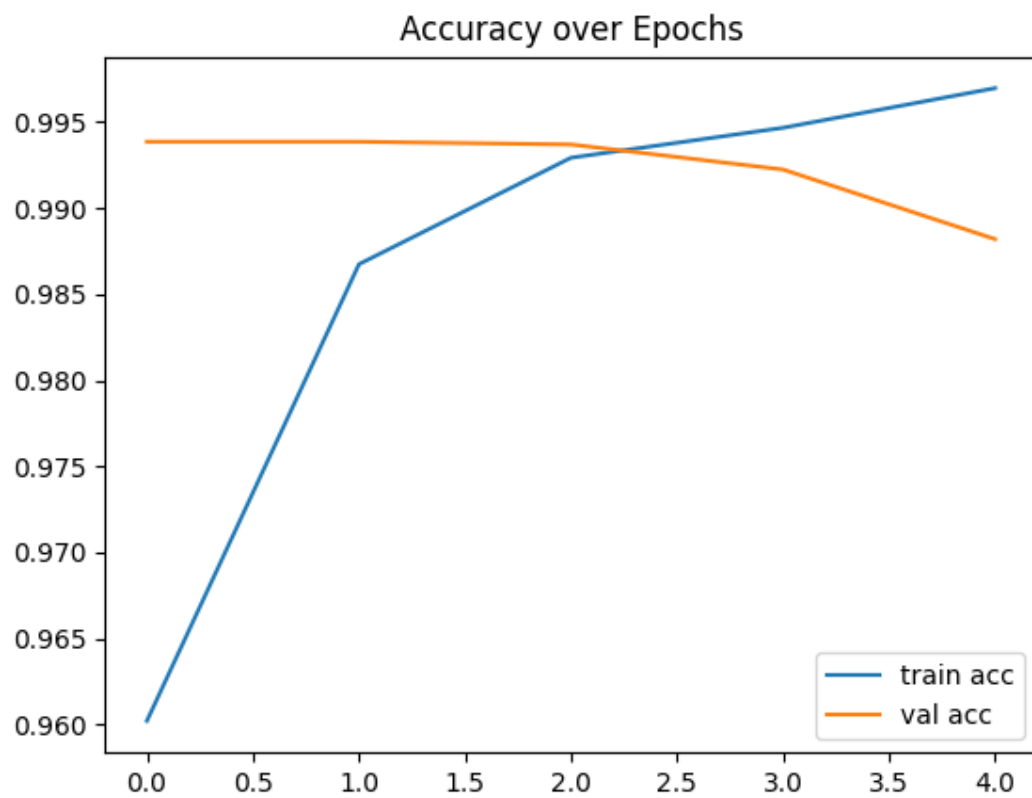
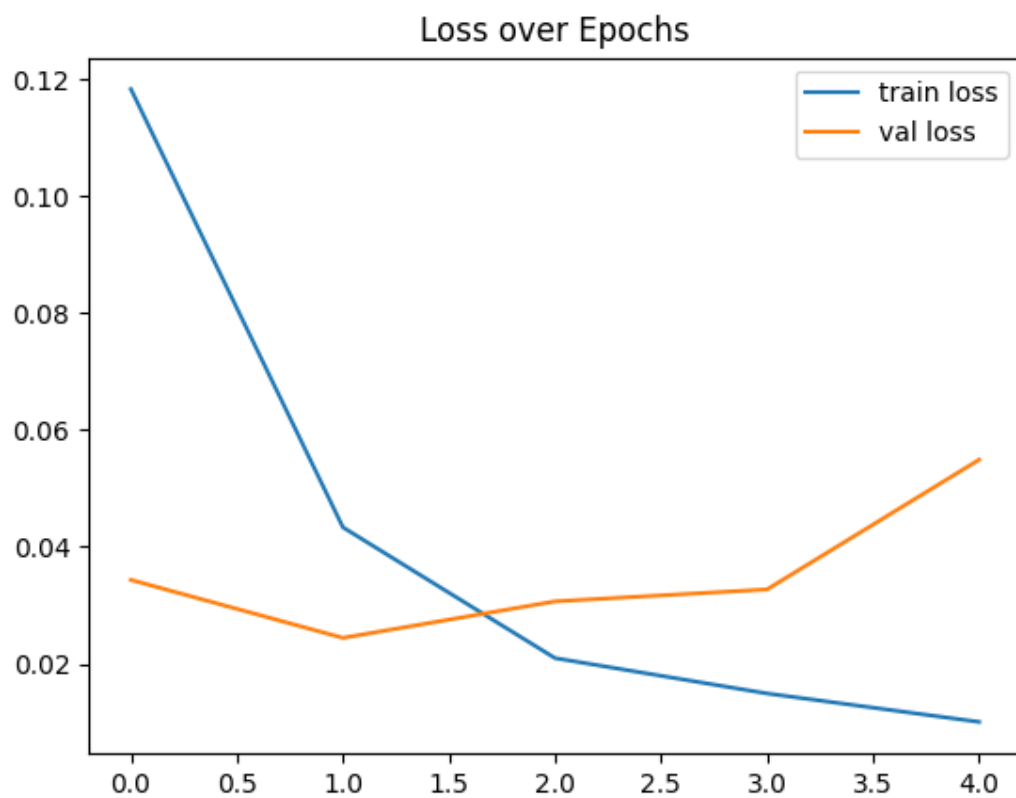


Figure 8.7: Accuracy Plot



CHAPTER 9

CONCLUSIONS

9.1 Conclusion

The proposed lab management desktop application will offer a cutting-edge solution that addresses the limitations of current systems. It will provide cross-platform support, allowing seamless integration across various operating systems, and will aggregate real-time data from multiple computers to a central server for unified monitoring. Through its predictive maintenance features, the system will enable administrators to foresee potential system failures and take proactive measures to maintain optimal lab performance.

With advanced visualization and reporting tools, administrators will be able to identify trends, optimize resource allocation, and make data-driven decisions. The system's user-friendly interface will ensure that lab management tasks are handled efficiently, even by those with limited technical expertise. As the lab grows, the application will scale to accommodate increasing data volumes and evolving needs, making it a valuable, long-term solution for efficient and effective lab management.

9.2 Future Work

- **Scalability and Flexibility:** The current system is tailored to specific lab environments. Future versions could focus on enhancing scalability and flexibility by incorporating features that cater to different lab sizes, configurations, and industry requirements, making the application adaptable for a wider range of users.
- **Integration with Cloud Services:** To enhance accessibility and data storage, future development could integrate cloud-based solutions for data management. This would allow administrators to access and manage lab data remotely, ensuring system performance and health monitoring beyond local networks.
- **Enhanced Predictive Maintenance Capabilities:** While the current system provides predictive maintenance, future updates could incorporate advanced machine learning algorithms. These algorithms would analyze historical data more effectively to predict system malfunctions or potential issues, improving maintenance accuracy and reducing downtime further.
- **User Training and Onboarding Modules:** Introducing training modules and onboarding guides for lab managers could improve the user experience and efficiency. These modules could provide real-time guidance and instructions on how to use the software effectively, reducing the learning curve for new users.
- **Mobile Application Development:** Developing a companion mobile application could enable administrators to monitor and manage lab activities on the go. This app could provide real-time notifications, system health alerts, and access to critical reports, enhancing the convenience and efficiency of lab management.

9.3 Applications

- **University Computer Labs:** In educational institutions, this system can be deployed in computer labs to monitor and manage computer usage, ensuring that

resources are used efficiently. Lab administrators can use the predictive maintenance feature to minimize downtime and maintain system performance, allowing students and faculty to have reliable access to lab resources for their academic and research needs.

- **Corporate Training Centers:** For businesses with in-house training facilities or labs, this application can monitor and optimize computer usage. The analytics and reporting features can help administrators understand usage patterns, allowing them to adjust schedules and resources accordingly. Predictive maintenance ensures that systems are operational during critical training sessions, improving overall training effectiveness and ensuring that learning environments are continuously available.
- **Healthcare Simulation Labs:** In medical training facilities, this system helps manage simulation labs where trainees use specialized software for practice. By monitoring system usage and health, lab managers can ensure that equipment is functional and ready for use. Predictive maintenance helps prevent unexpected failures, ensuring continuous operation of simulation tools that are vital for hands-on medical training and education.
- **Research and Development Facilities:** R&D labs, especially those involving high-performance computing, can benefit from this application by tracking system performance and usage. Lab administrators gain insights into how systems are being utilized, enabling them to optimize resource allocation for various projects. The predictive maintenance feature ensures that critical research tasks are not interrupted by system failures, enhancing overall research productivity.
- **Public Libraries and Community Centers:** This system can be used to manage and maintain computer resources in public libraries or community centers where multiple users access systems daily. By tracking usage patterns, administrators can determine peak usage times and allocate resources accordingly. The predictive maintenance feature allows for proactive system management, reducing the risk of computer downtime, which is essential for maintaining public access to technology and information.

CHAPTER 10

APPENDIX

10.1 Appendix A: Problem Statement Feasibility Assessment

The proposed hybrid SQL injection detection system is evaluated for feasibility as follows:

- **Satisfiability:** The system requirements can be met using current, mature technologies:
 - *Rule-based validation* implemented in Python/Flask for syntax and metadata checks.
 - *Deep learning classification* via a pre-trained TensorFlow/Keras LSTM model.
 - *Web frontend* using React.js for real-time query submission and dashboard visualization.
 - *Data storage* in MySQL (metadata and query logs).

Thus, a real-time detection pipeline with end-to-end latency $\leq 150\text{ms}$ is achievable.

- **Complexity Classification:**
 - *Rule-based checks* (balanced parentheses/quotes, schema lookups) run in $O(k)$ time per query, where k is query length or number of referenced identifiers.
 - *LSTM inference* is $O(n \cdot d)$ per query, where n is sequence length and d is the LSTM hidden dimension—both bounded constants in practice.
 - No exponential search or combinatorial optimization is required; all operations lie in polynomial time (**P**-type).
- **Mathematical Representation:** As defined in Section 4.2, the system is modeled by the tuple

$$S = \{I, V, C, L, D, A, R\},$$

where:

- I = set of input SQL queries.
 - V = rule-based validator $I \rightarrow \{0, 1\}$.
 - C = LSTM classifier $I \rightarrow [0, 1]$.
 - L = decision function combining V and C .
 - D = MySQL database.
 - A = actors {User, Admin}.
 - R = rules for blocking or allowing queries.
- **Modern Algebra Feasibility:** The system's modules and transitions can be abstracted as a finite state machine or function composition:

$$f: I \xrightarrow{V} \{0, 1\} \xrightarrow{L} \{\text{safe}, \text{injection}\},$$

with C integrated into L . All mappings are deterministic or computed in constant/polynomial time, ensuring functional completeness and logical implementability.

Therefore, the hybrid SQL injection detection problem is both satisfiable with existing tools and solvable in polynomial time; it does not reduce to any known NP-Complete or NP-Hard problem class.“

10.2 Appendix B: Paper Publication Details

- **Title:** Data Leak Detection Due to SQL Injection
- **Name of Conference:** International Journal of Scientific Research in Engineering And Management - (IJSREM)

10.3 Reviewer's Comments

- “Well-defined scope and clearly modular architecture separating rule-based validation and LSTM-based detection.”
 - “Real-time classification and alerting mechanism demonstrates practical applicability for database-centric applications.”
 - “Could further explore scalability under high query load and integration of anomaly detection for zero-day injection patterns.”
 - “The paper introduces a unique hybrid approach combining deterministic SQL validation with deep learning for robust SQL injection detection.”
 - “Support for multiple database engines (e.g. MySQL, PostgreSQL) and centralized log aggregation enhances the solution's versatility.”
 - “Interactive React.js dashboard with Chart.js visualizations provides valuable real-time feedback for security analysts.”
 - “A plagiarism check is recommended to confirm the originality of the proposed methodology and textual content.”
 - “Needs more detail on dataset composition (sources, size, class balance) and performance evaluation under varied workloads.”
 - “Data privacy, secure storage, and encryption measures should be discussed in greater depth to address compliance and confidentiality concerns.”
 - “A comparative analysis with existing SQL injection detection tools (e.g. static code analyzers, WAF modules) would strengthen the paper's contribution.”
- **Certificate:** Attached as Appendix B.1
 - **Published Paper:** Attached as Appendix B.2

10.4 Appendix C: Plagiarism Report

Tool Used: <https://plagiarismdetector.net/>

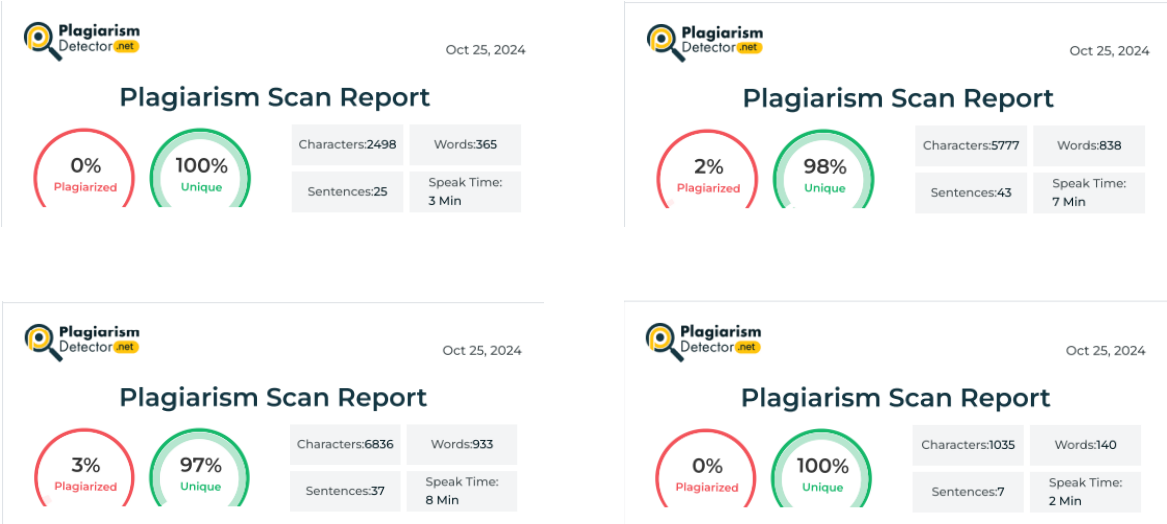


Figure 10.1: Plagiarism Report Snapshots for Various Project Sections

Thus, the overall plagiarism percentage is **2%**.

CHAPTER 11

REFERENCES

- [1] M. Bishop, *Computer Security: Art and Science*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2018.
- [2] OWASP Foundation, "OWASP Top 10- Injection," OWASP, 2021.
- [3] W. G. J. Halfond, J. Viegas, and A. Orso, "A classification of SQL injection attacks and countermeasures," in *Proc. IEEE Int. Symp. Secure Softw. Eng.*, 2006.
- [4] Y. Kim, H. Kim, and Y. Kim, "LSTM-based detection of SQL injection attacks," *Int. J. Comput. Appl.*, vol. 177, no. 24, pp. 10–15, 2020.
- [5] D. Ucci, L. Aniello, and R. Baldoni, "Survey of machine learning techniques for malware analysis," *Computers Security*, vol. 81, pp. 123–147, Mar. 2019.
- [6] K. Tasdemir, M. Cetin, and S. Arslan, "Advancing SQL Injection Detection for High-Speed Data Centers: A Novel Approach Using Cascaded NLP," *arXiv*, Dec. 2023.
- [7] N. S. Dasari and R. Ranjan, "Enhancing SQL Injection Detection and Prevention Using Generative Models," *arXiv*, Feb. 2025.
- [8] Q. Zhao and X. Li, "Deep Learning for SQL Injection Detection: Leveraging CNN and LSTM Networks," *arXiv*, Jul. 2024.
- [9] L. Zhang and Y. Liu, "A Novel Hybrid Approach for SQL Injection Detection Using Random Forest and Deep Neural Networks," *arXiv*, Mar. 2025.
- [10] R. Patel and V. Mehta, "Graph-Based Detection of SQL Injection Using Context-Aware Query Representation," *arXiv*, May 2024.
- [11] T. Roy and P. Dutta, "API-Level SQL Injection Detection Using Transformer-Based Models," *arXiv*, Jan. 2025.
- [12] A. Singh and D. Rao, "Real-Time SQL Injection Prevention in DevSec Ops Using Reinforcement Learning," *arXiv*, Sep. 2024.